Python Programming Lab Record

Revised edition: boxed test cases and larger, neatly framed sample-run screenshots.

Program 1: Hospital Billing and Patient Classification

1. Problem Statement

Calculate a patient bill using consultation, diagnostic and treatment charges, give a senior discount and classify payment.

2. Objectives

Implement the stated calculation using basic Python input, arithmetic, conditions and (where required) loops; display a clear result.

3. Assumptions

Charges and age are nonnegative; senior age begins at 60; low payment is at most Rs. 5000.

4. Algorithm

Add charges; if age >= 60 apply 10% discount; display total; classify total <= 5000 as low, otherwise high.

5. Flowchart

START -> Read inputs -> Process values -> Apply conditions / loop -> Display result -> END

6. Pseudocode

```
BEGIN
  Add charges
  If age >= 60 apply 10% discount
  Display total
  Classify total <= 5000 as low, otherwise high.
END
```

7. Python Source Code

```
consultation = int(input("Enter consultation charge: "))
diagnostic = int(input("Enter diagnostic charge: "))
treatment = int(input("Enter treatment charge: "))
age = int(input("Enter patient age: "))
total = consultation + diagnostic + treatment
if age >= 60:
    total = total - total * 0.10
print("Total Bill:", total)
if total <= 5000:



          2
    print("Low Payment Category")
else:
    print("High Payment Category")
```

8. Test Cases

Test	Input	Expected output
1	1000, 2000, 3000, age 65	5400.0; High Payment Category
2	1000, 1000, 1000, age 30	3000; Low Payment Category

9. Output (Sample Run Screenshot)

```
Python 3 - Sample Run
Inputs: consultation=1000, diagnostic=2000, treatment=3000, age=65

Total Bill: 5400.0
High Payment Category
```

10. Conclusion

The program demonstrates basic Python input, calculations and decision-making and produces the expected result for the illustrated test case.

Program 2: University Scholarship Eligibility

1. Problem Statement

Check whether a student meets marks, attendance and income criteria.

2. Objectives

Implement the stated calculation using basic Python input, arithmetic, conditions and (where required) loops; display a clear result.

3. Assumptions

Marks and attendance are percentages; income is annual rupees; all three conditions must hold.

4. Algorithm

Read marks, attendance and income; test all three eligibility conditions; print result.

5. Flowchart

START -> Read inputs -> Process values -> Apply conditions / loop -> Display result -> END

6. Pseudocode

```
BEGIN
  Read marks, attendance and income
  Test all three eligibility conditions
  Print result.
END
```

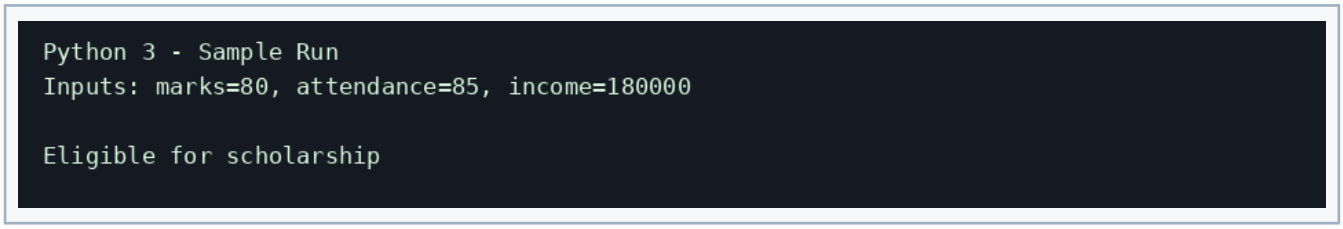
7. Python Source Code

```
marks = int(input("Enter aggregate marks: "))
attendance = int(input("Enter attendance percentage: "))
income = int(input("Enter annual family income: "))
if marks >= 75 and attendance >= 75 and income <= 200000:
    print("Eligible for scholarship")
else:
    print("Not eligible for scholarship")
```

8. Test Cases

Test	Input	Expected output
1	Marks 80; attendance 85; income 150000	Eligible for scholarship
2	Marks 70; attendance 90; income 100000	Not eligible for scholarship

9. Output (Sample Run Screenshot)



10. Conclusion

The program demonstrates basic Python input, calculations and decision-making and produces the expected result for the illustrated test case.

Program 3: Employee Salary Calculation

1. Problem Statement

Calculate gross salary, percentage deduction and net salary.

2. Objectives

Implement the stated calculation using basic Python input, arithmetic, conditions and (where required) loops; display a clear result.

3. Assumptions

All salary inputs are nonnegative; gross <= 20000 has 5% deduction, otherwise 10%.

4. Algorithm

Add four salary components; choose 5% or 10% deduction by gross; subtract and display.

5. Flowchart

START -> Read inputs -> Process values -> Apply conditions / loop -> Display result -> END

6. Pseudocode

```
BEGIN
  Add four salary components
  Choose 5% or 10% deduction by gross
  Subtract and display.
END
```

7. Python Source Code

```
basic = int(input("Enter basic salary: "))
allowance = int(input("Enter allowance: "))
overtime = int(input("Enter overtime pay: "))
incentive = int(input("Enter incentive: "))
gross = basic + allowance + overtime + incentive
if gross <= 20000:
    deduction = gross * 0.05
else:
    deduction = gross * 0.10
net = gross - deduction
print("Gross Salary:", gross)
print("Deduction:", deduction)


          6
print("Net Salary:", net)
```

8. Test Cases

Test	Input	Expected output
1	Basic 15000; allowance 2000; overtime 1000; incentive 1000	Gross 19000; deduction 950.0; net 18050.0
2	Basic 20000; allowance 5000; overtime 0; incentive 0	Gross 25000; deduction 2500.0; net 22500.0

9. Output (Sample Run Screenshot)

```
Python 3 - Sample Run
Inputs: basic=18000, allowance=2000, overtime=1000, incentive=1000

Gross Salary: 22000
Deduction: 2200.0
Net Salary: 19800.0
```

10. Conclusion

The program demonstrates basic Python input, calculations and decision-making and produces the expected result for the illustrated test case.

Program 4: Mobile Data Plan Selection

1. Problem Statement

Recommend a plan based on usage and available budget.

2. Objectives

Implement the stated calculation using basic Python input, arithmetic, conditions and (where required) loops; display a clear result.

3. Assumptions

Usage is GB and nonnegative; budget is rupees; plan capacities are 2, 5 and 10 GB.

4. Algorithm

Read usage and budget; test Plan 1, then Plan 2, then Plan 3; otherwise show no suitable plan.

5. Flowchart

START -> Read inputs -> Process values -> Apply conditions / loop -> Display result -> END

6. Pseudocode

```
BEGIN
  Read usage and budget
  Test plan 1, then plan 2, then plan 3
  Otherwise show no suitable plan.
END
```

7. Python Source Code

```
usage = int(input("Enter data usage in GB: "))
budget = int(input("Enter your budget: "))
plan1 = 199
plan2 = 299
plan3 = 399
if usage <= 2 and budget >= plan1:
    print("Best Plan: Plan 1 - Rs.199")
elif usage <= 5 and budget >= plan2:
    print("Best Plan: Plan 2 - Rs.299")
elif usage <= 10 and budget >= plan3:
    print("Best Plan: Plan 3 - Rs.399")
else:


          8
    print("No suitable plan available")
```

8. Test Cases

Test	Input	Expected output
1	Usage 2 GB; budget 200	Plan 1 - Rs.199
2	Usage 8 GB; budget 300	No suitable plan available

9. Output (Sample Run Screenshot)

```
Python 3 - Sample Run
Inputs: usage=4, budget=350

Best Plan: Plan 2 - Rs.299
```

10. Conclusion

The program demonstrates basic Python input, calculations and decision-making and produces the expected result for the illustrated test case.

Program 5: Railway Fare Calculation

1. Problem Statement

Calculate fare from distance and class, with senior discount and booking charge.

2. Objectives

Implement the stated calculation using basic Python input, arithmetic, conditions and (where required) loops; display a clear result.

3. Assumptions

Sleeper costs Rs.2/km, AC Rs.5/km; seniors 60+ receive 20% discount before Rs.50 booking charge.

4. Algorithm

Determine base fare from class; reject invalid class; discount seniors; add Rs.50; display fare.

5. Flowchart

START -> Read inputs -> Process values -> Apply conditions / loop -> Display result -> END

6. Pseudocode

```
BEGIN
  Determine base fare from class
  Reject invalid class
  Discount seniors
  Add rs.50
  Display fare.
END
```

7. Python Source Code

```
distance = int(input("Enter distance in km: "))
age = int(input("Enter passenger age: "))
travel_class = input("Enter class (sleeper/AC): ")
if travel_class == "sleeper":
    fare = distance * 2
elif travel_class == "AC":
    fare = distance * 5
else:
    fare = 0
if age >= 60 and fare > 0:


        10
    fare = fare * 0.80
if fare > 0:
    fare = fare + 50
    print("Total Railway Fare:", fare)
else:
    print("Invalid travel class")
```

8. Test Cases

Test	Input	Expected output
1	100 km; age 65; sleeper	Total Railway Fare: 210.0
2	100 km; age 30; AC	Total Railway Fare: 550

9. Output (Sample Run Screenshot)

```
Python 3 - Sample Run
Inputs: distance=100, age=65, travel_class=AC

Total Railway Fare: 450.0
```

10. Conclusion

The program demonstrates basic Python input, calculations and decision-making and produces the expected result for the illustrated test case.

Program 6: Shopping Bill with Discount and Tax

1. Problem Statement

Calculate product total, tiered discount, tax and final bill.

2. Objectives

Implement the stated calculation using basic Python input, arithmetic, conditions and (where required) loops; display a clear result.

3. Assumptions

Price and quantity are nonnegative; discount is 10% at Rs.5000+, else 5%; tax is 5% after discount.

4. Algorithm

Multiply price by quantity; choose discount; compute tax on discounted subtotal; display bill.

5. Flowchart

START -> Read inputs -> Process values -> Apply conditions / loop -> Display result -> END

6. Pseudocode

```
BEGIN
  Multiply price by quantity
  Choose discount
  Compute tax on discounted subtotal
  Display bill.
END
```

7. Python Source Code

```
price = int(input("Enter product price: "))
quantity = int(input("Enter quantity: "))
total = price * quantity
if total >= 5000:
    discount = total * 0.10
else:
    discount = total * 0.05
tax = (total - discount) * 0.05
bill = total - discount + tax
print("Total Amount:", total)
print("Discount:", discount)


        12
print("Tax:", tax)
print("Final Bill:", bill)
```

8. Test Cases

Test	Input	Expected output
1	Price 1000; quantity 5	Total 5000; discount 500.0; tax 225.0; bill 4725.0
2	Price 100; quantity 2	Total 200; discount 10.0; tax 9.5; bill 199.5

9. Output (Sample Run Screenshot)

```
Python 3 - Sample Run
Inputs: price=1000, quantity=6

Total Amount: 6000
Discount: 600.0
Tax: 270.0
Final Bill: 5670.0
```

10. Conclusion

The program demonstrates basic Python input, calculations and decision-making and produces the expected result for the illustrated test case.

Program 7: Manufacturing Profit Target

1. Problem Statement

Calculate manufacturing cost and profit, then assess target achievement.

2. Objectives

Implement the stated calculation using basic Python input, arithmetic, conditions and (where required) loops; display a clear result.

3. Assumptions

All amounts are rupees; profit = selling price minus total cost; negative profit represents loss.

4. Algorithm

Sum costs; subtract from selling price; compare profit against target; report.

5. Flowchart

START -> Read inputs -> Process values -> Apply conditions / loop -> Display result -> END

6. Pseudocode

```
BEGIN
  Sum costs
  Subtract from selling price
  Compare profit against target
  Report.
END
```

7. Python Source Code

```
material = int(input("Enter material cost: "))
labour = int(input("Enter labour cost: "))
machine = int(input("Enter machine cost: "))
selling = int(input("Enter selling price: "))
target = int(input("Enter profit target: "))
cost = material + labour + machine
profit = selling - cost
print("Total Cost:", cost)
print("Profit:", profit)
if profit >= target:
    print("Profit Target Achieved")


        14
else:
    print("Profit Target Not Achieved")
```

8. Test Cases

Test	Input	Expected output
1	Material 1000; labour 500; machine 500; selling 3000; target 800	Cost 2000; profit 1000; achieved
2	Material 1000; labour 500; machine 500; selling 2300; target 500	Cost 2000; profit 300; not achieved

9. Output (Sample Run Screenshot)

```
Python 3 - Sample Run
Inputs: material=2000, labour=1000, machine=500, selling=5000, target=1000

Total Cost: 3500
Profit: 1500
Profit Target Achieved
```

10. Conclusion

The program demonstrates basic Python input, calculations and decision-making and produces the expected result for the illustrated test case.

Program 8: Travel Currency Budget

1. Problem Statement

Convert dollar expenses to rupees, add conversion fee and compare with budget.

2. Objectives

Implement the stated calculation using basic Python input, arithmetic, conditions and (where required) loops; display a clear result.

3. Assumptions

Exchange rate means INR per USD; fee and budget are INR; values are nonnegative.

4. Algorithm

Read expense, exchange rate, fee and budget; calculate converted total; compare to budget.

5. Flowchart

START -> Read inputs -> Process values -> Apply conditions / loop -> Display result -> END

6. Pseudocode

```
BEGIN
  Read expense, exchange rate, fee and budget
  Calculate converted total
  Compare to budget.
END
```

7. Python Source Code

```
usd = float(input("Enter expenses in dollars: "))
rate = float(input("Enter exchange rate: "))
fee = float(input("Enter conversion fee: "))
budget = float(input("Enter budget in rupees: "))
total = usd * rate + fee
print("Total Travel Cost:", total)
if total <= budget:
    print("Within Budget")
else:
    print("Budget Exceeded")
```

8. Test Cases

Test	Input	Expected output
1	USD 100; rate 83; fee 100; budget 9000	Total Travel Cost: 8400.0; Within Budget
2	USD 100; rate 83; fee 100; budget 8000	Total Travel Cost: 8400.0; Budget Exceeded

9. Output (Sample Run Screenshot)

```
Python 3 - Sample Run
Inputs: usd=100, rate=85, fee=100, budget=9000

Total Travel Cost: 8600.0
Within Budget
```

10. Conclusion

The program demonstrates basic Python input, calculations and decision-making and produces the expected result for the illustrated test case.

Program 9: College Event Cost and Registrations

1. Problem Statement

Calculate event expenses and the number of paid student registrations required after sponsorship.

2. Objectives

Implement the stated calculation using basic Python input, arithmetic, conditions and (where required) loops; display a clear result.

3. Assumptions

All costs are rupees; each student pays the same positive fee; registrations are rounded up.

4. Algorithm

Sum event expenses; subtract sponsorship; if covered report so; otherwise divide remaining by positive fee and round up.

5. Flowchart

START -> Read inputs -> Process values -> Apply conditions / loop -> Display result -> END

6. Pseudocode

```
BEGIN
  Sum event expenses
  Subtract sponsorship
  If covered report so
  Otherwise divide remaining by positive fee and round up.
END
```

7. Python Source Code

```
venue = int(input("Enter venue cost: "))
equipment = int(input("Enter equipment cost: "))
food = int(input("Enter catering cost: "))
transport = int(input("Enter transport cost: "))
sponsor = int(input("Enter sponsorship amount: "))
fee = int(input("Enter registration fee per student: "))
total = venue + equipment + food + transport
remaining = total - sponsor
print("Total Event Cost:", total)



        18
if remaining <= 0:
    print("Event Cost Covered by Sponsorship")
elif fee > 0:
    students = (remaining + fee - 1) // fee
    print("Students Needed:", students)
else:
    print("Registration Fee Must Be Greater Than Zero")
```

8. Test Cases

Test	Input	Expected output
1	Venue 1000; equipment 500; food 1000; transport 500; sponsor 1000; fee 200	Total Event Cost: 3000; Students Needed: 10

Test	Input	Expected output
2	Same costs; sponsor 3000; fee 200	Event Cost Covered by Sponsorship

9. Output (Sample Run Screenshot)

```
Python 3 - Sample Run
Inputs: venue=5000, equipment=2000, food=3000, transport=1000, sponsor=5000, fee=200

Total Event Cost: 11000
Students Needed: 30
```

10. Conclusion

The program demonstrates basic Python input, calculations and decision-making and produces the expected result for the illustrated test case.

Program 10: Student Result with Grace Marks

1. Problem Statement

Process multiple subject marks, apply eligible grace marks, and classify result.

2. Objectives

Implement the stated calculation using basic Python input, arithmetic, conditions and (where required) loops; display a clear result.

3. Assumptions

Subject count is positive; marks range 0-100; grace is applied separately only when it raises a failed subject to 35; percentage is average.

4. Algorithm

Read count and grace; loop through marks; apply grace where it enables pass; flag failures; compute average; classify.

5. Flowchart

START -> Read inputs -> Process values -> Apply conditions / loop -> Display result -> END

6. Pseudocode

```
BEGIN
  Read count and grace
  Loop through marks
  Apply grace where it enables pass
  Flag failures
  Compute average
  Classify.
END
```

7. Python Source Code

```
subjects = int(input("Enter number of subjects: "))
grace = int(input("Enter grace marks: "))
total = 0
failed = False
for i in range(subjects):
    marks = int(input("Enter subject marks: "))



         20
    if marks < 35 and marks + grace >= 35:
        marks = 35
    if marks < 35:
        failed = True
    total += min(marks, 100)
percentage = total / subjects
if failed:
    print("Result: Fail")
elif percentage >= 75:
    print("Result: Distinction")
elif percentage >= 60:
    print("Result: First Class")
else:
    print("Result: Pass")
```

8. Test Cases

Test	Input	Expected output
1	3 subjects: 80, 70, 90; grace 5	Result: Distinction
2	3 subjects: 30, 50, 60; grace 5	Result: Pass

9. Output (Sample Run Screenshot)

```
Python 3 - Sample Run
Inputs: subjects=3, grace=3, marks=[32,80,90]

Result: First Class
```

10. Conclusion

The program demonstrates basic Python input, calculations and decision-making and produces the expected result for the illustrated test case.